

LiDAR Patrol

LiDAR Patrol

1. Course Content
2. Demo
3. Source Code Analysis

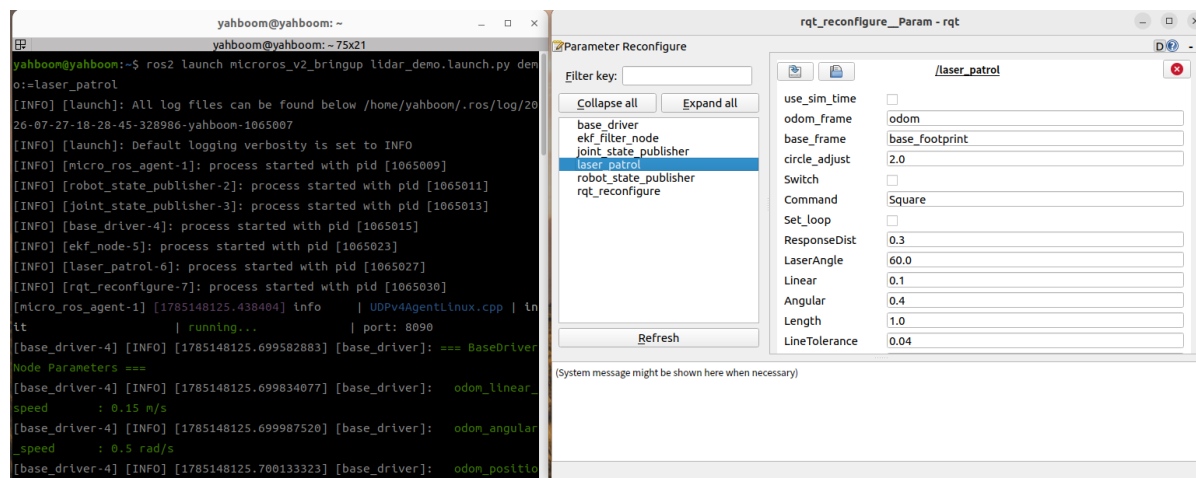
1. Course Content

- Run the demo program, set the patrol route in the dynamic parameter tuner, then click start. The robot moves along the configured patrol route, while the LiDAR scans for obstacles within the configured laser angle and obstacle detection distance. If an obstacle is present, the robot stops and the buzzer sounds; if there is no obstacle, it continues patrolling.

2. Demo

```
ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_patrol
```

- Click the **laser_patrol** node in the left options panel of the **rqt_reconfigure** panel. If there is no node in the left options right after startup, click **Refresh** below to refresh



- Fill in the preset patrol route in **[Command]**, then tick Switch to start the robot's patrol state

use_sim_time	☐
odom_frame	odom
base_frame	base_footprint
circle_adjust	2.0
Switch	☐
Command	LengthTest
Set_loop	☐

[!TIP]

rqt_reconfigure panel parameter descriptions:

- **odom_frame**: name of the odometry coordinate frame

- `base_frame`: name of the base coordinate frame
- `circle_adjust`: when the patrol route is a circle, this value can be used as a coefficient to adjust the circle size; see the code for details
- `Switch`: play mode switch
- `Command`: patrol route, with the following options: [LengthTest] - line patrol, [Circle] - circle patrol, [Square] - square patrol, [Triangle] - triangle patrol.
- `Set_loop`: re-patrol switch; when set, the robot will continue looping patrol along the set route
- `ResponseDist`: obstacle detection distance
- `LaserAngle`: laser detection angle
- `Linear`: linear velocity magnitude
- `Angular`: angular velocity magnitude
- `Length`: straight-line motion distance
- `RotationTolerance`: allowed tolerance for rotation error
- `LineTolerance`: allowed tolerance for straight-line motion error

- After setting the route in [Command], click Switch to start patrolling. The terminal prints the following information:
- If there is an obstacle on the patrol path, the robot stops and the terminal displays **obstacles**

3. Source Code Analysis

Source code path:

```
$HOME/microros_ws/src/common_demo/common_demo/lidar/laser_patrol.py
```

The patrol node uses a **timer-driven + state machine** architecture, with the following core logic:

1. **Dynamic parameter loading:** At the beginning of each cycle, the `on_timer` callback reads all configurations from the parameter server (`Switch`, `Command`, `ResponseDist`, `Linear`, `Angular`, `Length`, `LaserAngle`, `LineTolerance`, `RotationTolerance`, etc.), supporting real-time adjustment via `rqt_reconfigure`.
2. **Patrol route dispatch:** When `Switch=True`, the corresponding patrol mode is selected according to the `Command` string:
 - `LengthTest`: line patrol, calls `advancing(self.Length)` to move forward the specified distance
 - `Circle`: circle patrol, calls `spin(360)` to rotate 360° in place
 - `Square`: square patrol, calls `Square()` to execute the sequence of four sides and four corners
 - `Triangle`: triangle patrol, calls `Triangle()` to execute the sequence of three sides and three corners
3. **Task completion handling:** After each patrol function returns `True`, `Switch` is automatically set to `False` and `Command` to `"finish"`, entering the stopped state.
4. **Loop patrol logic:** When `Switch=False` and `Command="finish"`:
 - If `Set_loop=True`: restore `Command` to the original route (`command_src`), set `Switch` back to `True`, and continue the next patrol round
 - If `Set_loop=False`: remain in the stopped state without auto-restart

Obstacle avoidance mechanism: Inside each patrol function (`advancing`, `Spin`, `Square`, `Triangle`), laser detection is continuously performed. When the distance to an obstacle ahead is $\leq$ `ResponseDist` and the angle is within $\pm$ `LaserAngle`, the robot immediately stops and triggers the buzzer, then continues executing after the obstacle moves away.

```
def on_timer(self):
    # print("self.error: ",self.error)
    self.Switch =
self.get_parameter("Switch").get_parameter_value().bool_value
    self.Command =
self.get_parameter("Command").get_parameter_value().string_value
    self.Set_loop =
self.get_parameter("Set_loop").get_parameter_value().bool_value
    self.ResponseDist = (
self.get_parameter("ResponseDist").get_parameter_value().double_value
    )
    self.Linear =
self.get_parameter("Linear").get_parameter_value().double_value
    self.Angular =
self.get_parameter("Angular").get_parameter_value().double_value
    self.Length =
self.get_parameter("Length").get_parameter_value().double_value
    self.LaserAngle = (
        self.get_parameter("LaserAngle").get_parameter_value().double_value
    )
    self.LineTolerance = (
self.get_parameter("LineTolerance").get_parameter_value().double_value
    )
    self.RotationTolerance = (
self.get_parameter("RotationTolerance").get_parameter_value().double_value
    )
    self.RotationScaling = (
self.get_parameter("RotationScaling").get_parameter_value().double_value
    )

    index = 0

    if self.Switch == True:
        index = 0
        self.get_logger().info("Switch True")
        if self.Command == "LengthTest":
            self.command_src = "LengthTest"
            self.get_logger().info("LengthTest")
            advancing = self.advancing(self.Length)
            if advancing == True:
                self.Switch = rclpy.parameter.Parameter(
                    "Switch", rclpy.Parameter.Type.BOOL, False
                )
                all_new_parameters = [self.Switch]
                self.set_parameters(all_new_parameters)
                self.Command = rclpy.parameter.Parameter(
                    "Command", rclpy.Parameter.Type.STRING, "finish"
                )
```

```

        all_new_parameters = [self.Command]
        self.set_parameters(all_new_parameters)

    elif self.Command == "Circle":
        self.command_src = "Circle"
        spin = self.Spin(360)
        if spin == True:
            self.get_logger().info("spin done")
            # self.Command = "finish"
            self.Switch = rclpy.parameter.Parameter(
                "Switch", rclpy.Parameter.Type.BOOL, False
            )
            all_new_parameters = [self.Switch]
            self.set_parameters(all_new_parameters)
            self.Command = rclpy.parameter.Parameter(
                "Command", rclpy.Parameter.Type.STRING, "finish"
            )
            all_new_parameters = [self.Command]
            self.set_parameters(all_new_parameters)

    elif self.Command == "Square":
        self.command_src = "Square"
        square = self.Square()
        if square == True:
            self.Command = rclpy.parameter.Parameter(
                "Command", rclpy.Parameter.Type.STRING, "finish"
            )
            all_new_parameters = [self.Command]
            self.set_parameters(all_new_parameters)

    elif self.Command == "Triangle":
        self.command_src = "Triangle"
        triangle = self.Triangle()
        if triangle == True:
            self.Command = rclpy.parameter.Parameter(
                "Command", rclpy.Parameter.Type.STRING, "finish"
            )
            all_new_parameters = [self.Command]
            self.set_parameters(all_new_parameters)

    else:
        self.pub_cmdvel.publish(Twist())
        self.pub_beep.publish(UInt16(data=0))
        self.get_logger().info("Switch False")
        if self.Command == "finish":
            self.get_logger().info("finish")
            if self.Set_loop == True:
                self.get_logger().info("Continue")
                self.Command = rclpy.parameter.Parameter(
                    "Command", rclpy.Parameter.Type.STRING, self.command_src
                )
                all_new_parameters = [self.Command]
                self.set_parameters(all_new_parameters)
                self.Switch = rclpy.parameter.Parameter(
                    "Switch", rclpy.Parameter.Type.BOOL, True
                )
                all_new_parameters = [self.Switch]
                self.set_parameters(all_new_parameters)

```

```
else:
    self.get_logger().info("Not loop")
    self.Switch = rclpy.parameter.Parameter(
        "Switch", rclpy.Parameter.Type.BOOL, False
    )
    all_new_parameters = [self.Switch]
    self.set_parameters(all_new_parameters)
```